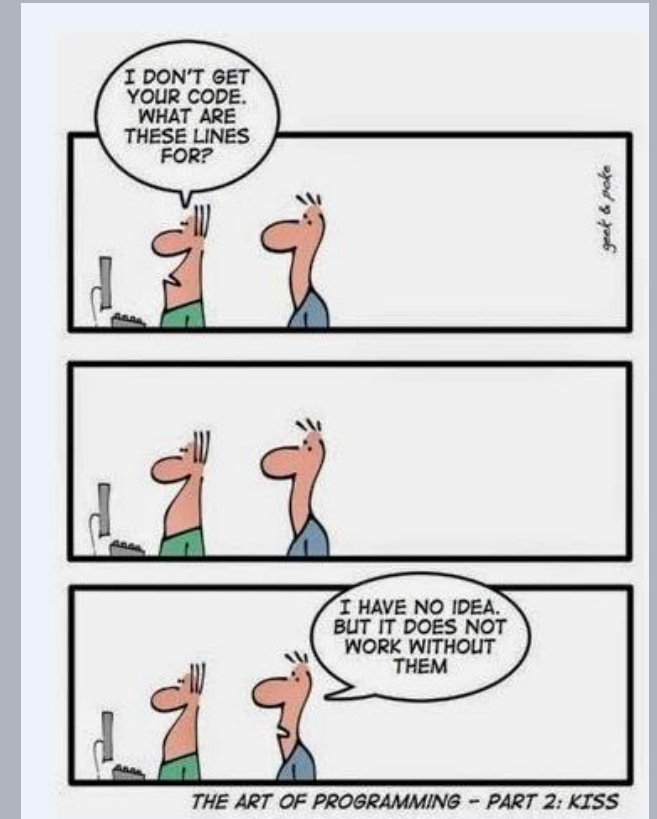


Advanced Software Design Techniques

Together, Doxygen, Models & Comments

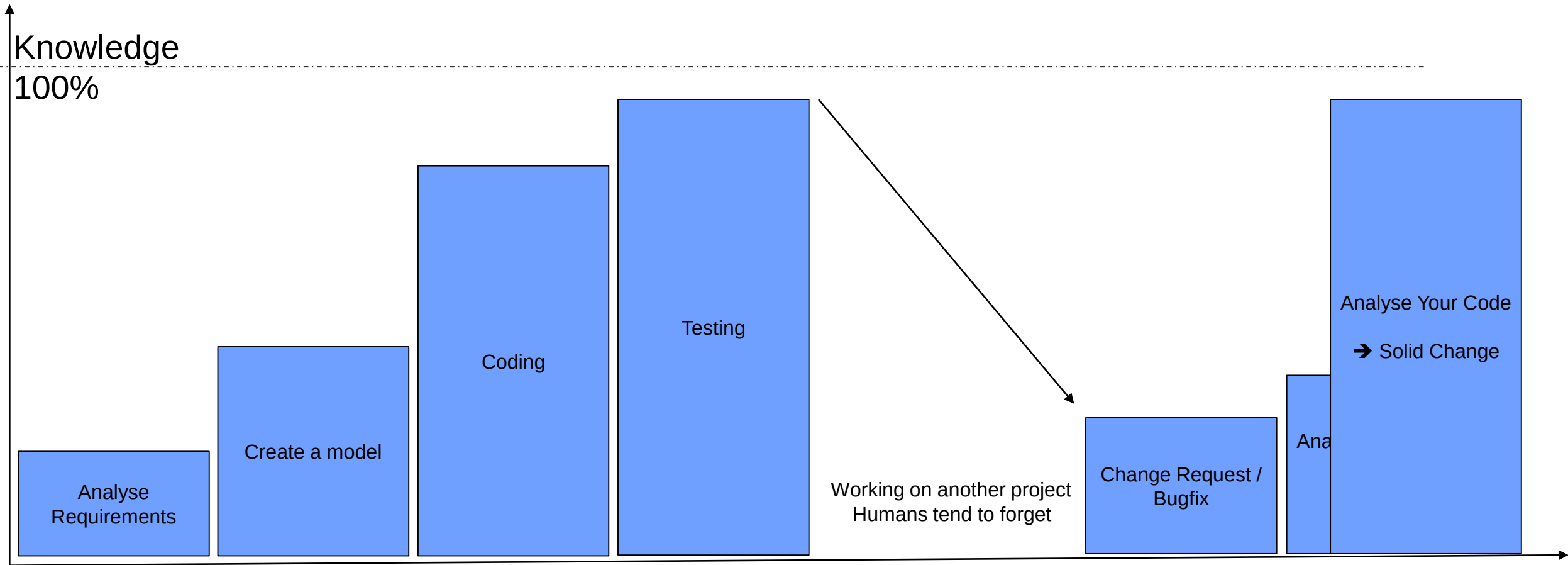
Prof. Dr.-Ing. Peter Fromm



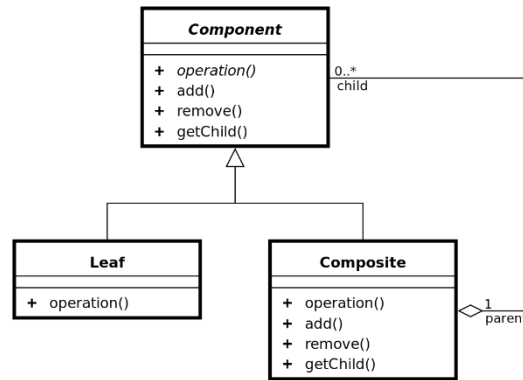
Content

- Motivation
- UML Models
- Using Together
- Using Doxygen

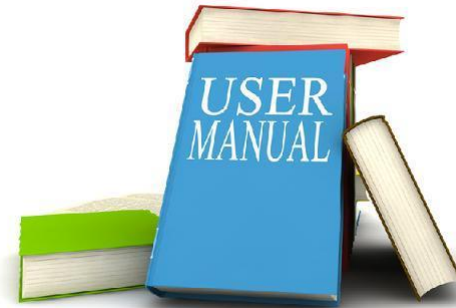
Why documentation, models and comments?



Theory

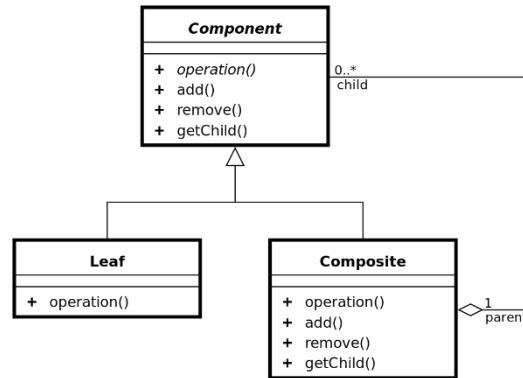


```
xTargetDistance = this->Hunlayer.TCreature.possx - u07->posx;
if ( xTargetDistance <= -1 )
    xTargetDistance = -(this->Hunlayer.TCreature.possx - u07->posx);
yTargetDistance = this->Hunlayer.TCreature.possy - u07->posy;
if ( yTargetDistance <= -1 )
    yTargetDistance = -(this->Hunlayer.TCreature.possy - u07->posy);
if ( xTargetDistance > 10 || yTargetDistance > 10 )
    goto LABEL_219;
if ( Strategy == 1 )
{
    SkillHealth = TSkill::Get((int)u07->baseclass_0_Skills[2]);
}
else
{
    if ( Strategy > 1 )
    {
        if ( Strategy == 2 )
        {
            CreatureDamage = TCombat::GetDamageByCreature(&this->Hunlayer.TCreature.Combat, u07->ID);
            goto LABEL_240;
        }
        CreatureDamage = 0;
        if ( Strategy == 3 )
        {
            LABEL_240:
            SomeRandom_99 = random(0, 99);
            if ( CreatureDamage > Goodness || CreatureDamage == Goodness && SomeRandom_99 > TieBreaker )
            {
                this->Target = u05;
                Goodness = CreatureDamage;
                TieBreaker = SomeRandom_99;
            }
            goto LABEL_219;
        }
    }
    LABEL_239:
    CreatureDamage = 0;
    error("Hunlayer::dieStimulus: Unbekannte Strategie %d\n", Strategy);
    goto LABEL_240;
    if ( Strategy )
    {
        goto LABEL_239;
    }
    SkillHealth = yTargetDistance * xTargetDistance;
    CreatureDamage = -SkillHealth;
    goto LABEL_240;
}
Sleep = 0;
goto LABEL_218;
}
TargetCreature = this->Target;
```



Upon release: All up
to date and in sync

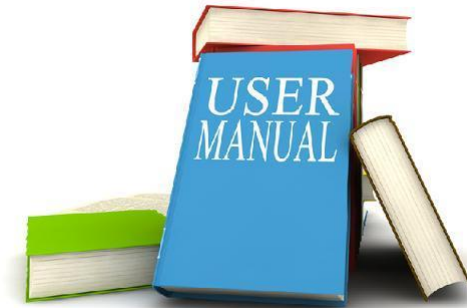
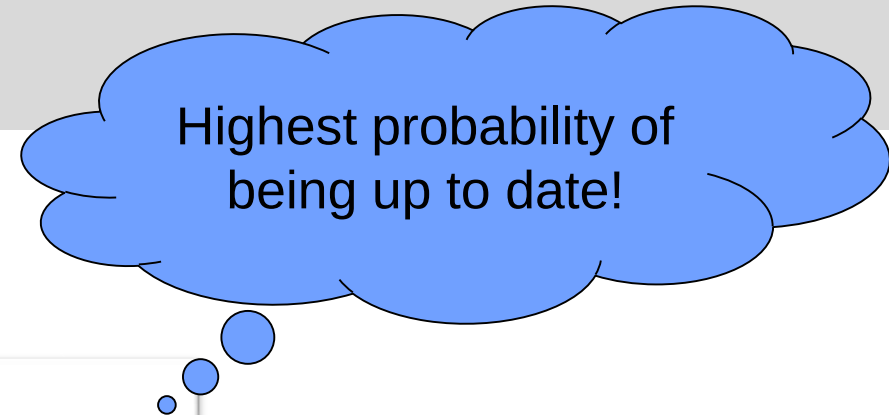
Code centric approach



Synchronise

```
IF ( sTargetDistance <= -1 )
    sTargetDistance = -(this->TNonplayer.TCreature.posex - u67->posex);
    gTargetDistance = this->TNonplayer.TCreature.posey - u67->posey;
    IF ( gTargetDistance <= -1 )
        gTargetDistance = -gTargetDistance;
    IF ( sTargetDistance > 10 || gTargetDistance > 10 )
        goto LABEL_219;
    IF ( Strategy == 1 )
    {
        SkillHealth = TSkill::Get((int)u67->baseclass_0.Skills[2]);
    }
    else
    {
        IF ( Strategy > 1 )
        {
            IF ( Strategy == 2 )
            {
                CreatureDamage = TCombat::GetDamageByCreature(&this->TNonplayer.TCreature.Combat, u67->10);
                goto LABEL_240;
            }
            CreatureDamage = 0;
            IF ( Strategy == 3 )
            {
                SomeRandom0_99 = random(0, 99);
                IF ( CreatureDamage > Goodness || CreatureDamage == Goodness && SomeRandom0_99 > TieBreaker )
                {
                    this->Target = u65;
                    Goodness = CreatureDamage;
                    TieBreaker = SomeRandom0_99;
                }
                goto LABEL_219;
            }
        }
        goto LABEL_239;
    }
    CreatureDamage = 0;
    error("TNonplayer::IdleStimulus: Unbekannte Strategie %d.\n", Strategy);
    IF ( Strategy )
        goto LABEL_239;
    SkillHealth = gTargetDistance * sTargetDistance;
    CreatureDamage = -SkillHealth;
    goto LABEL_240;
}
Sleep = 0;
goto LABEL_210;
```

Generate



This does not solve all problems!

Documentation

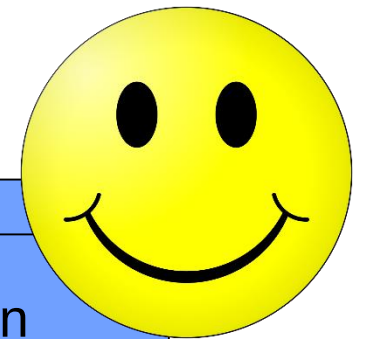
- Good for API description (Reference Manual)
- No chance to generate conceptual ideas leading to the code (User Manual)

Model

- Good for keeping names and dependencies in sync
- Abstract models describing the concept require human skills

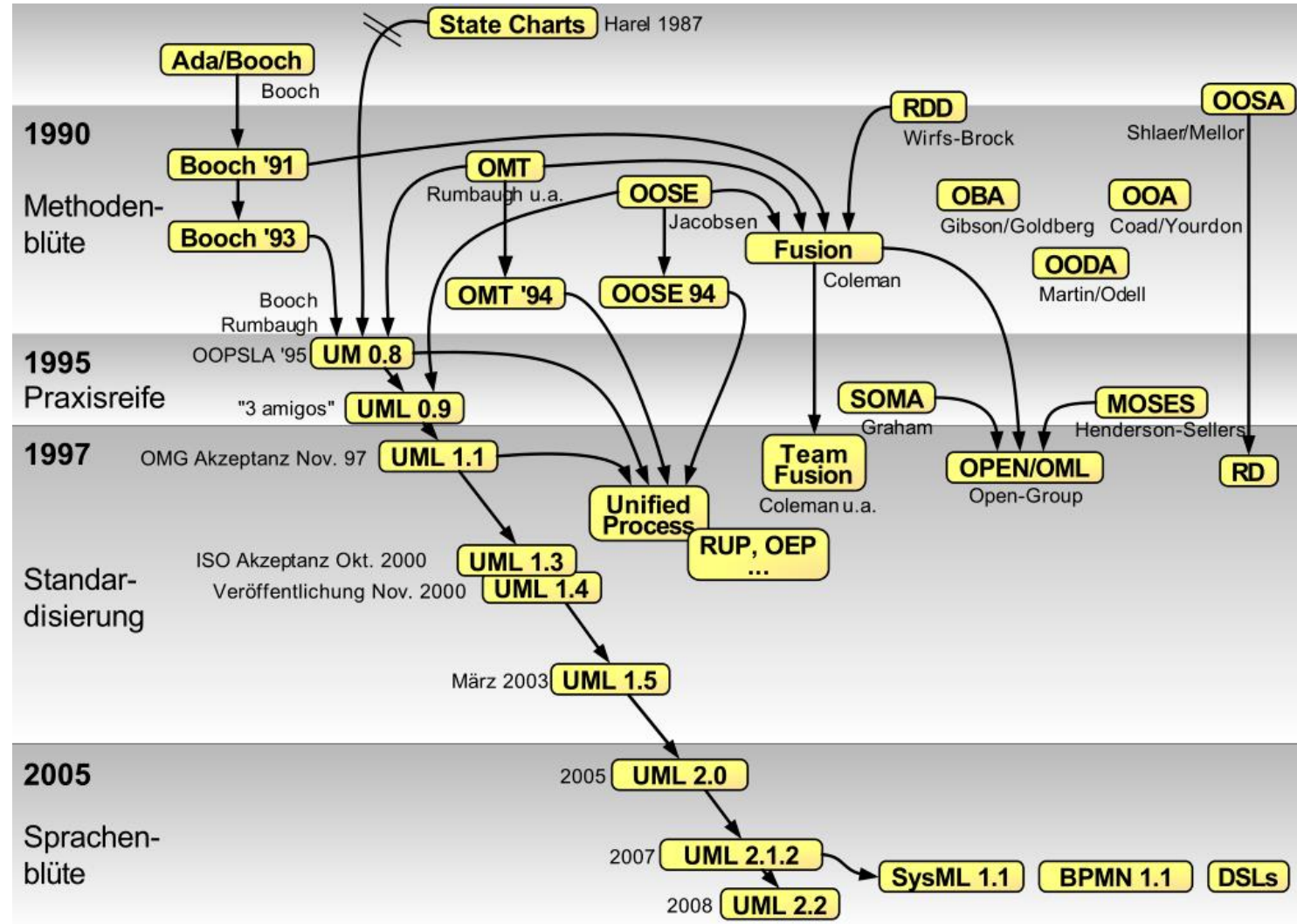


Autogenerate Reference
Manual

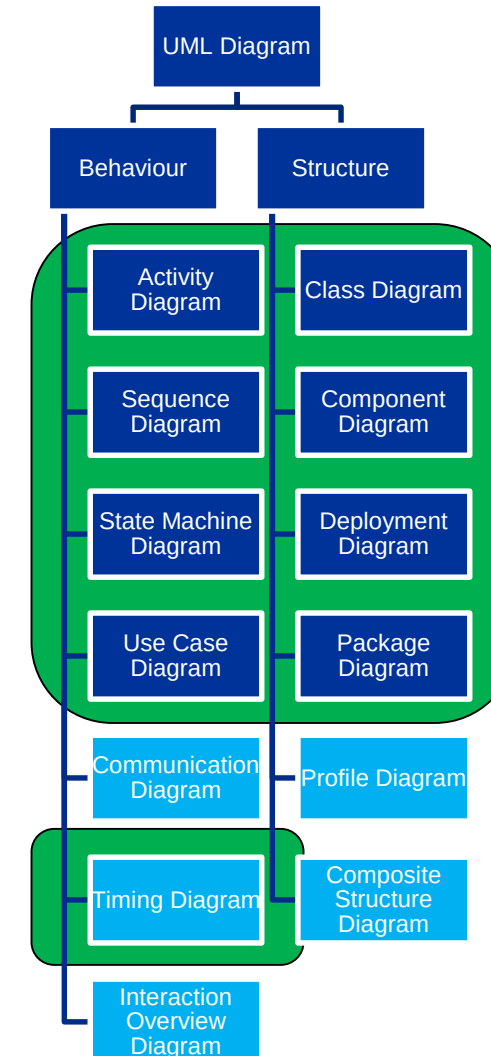
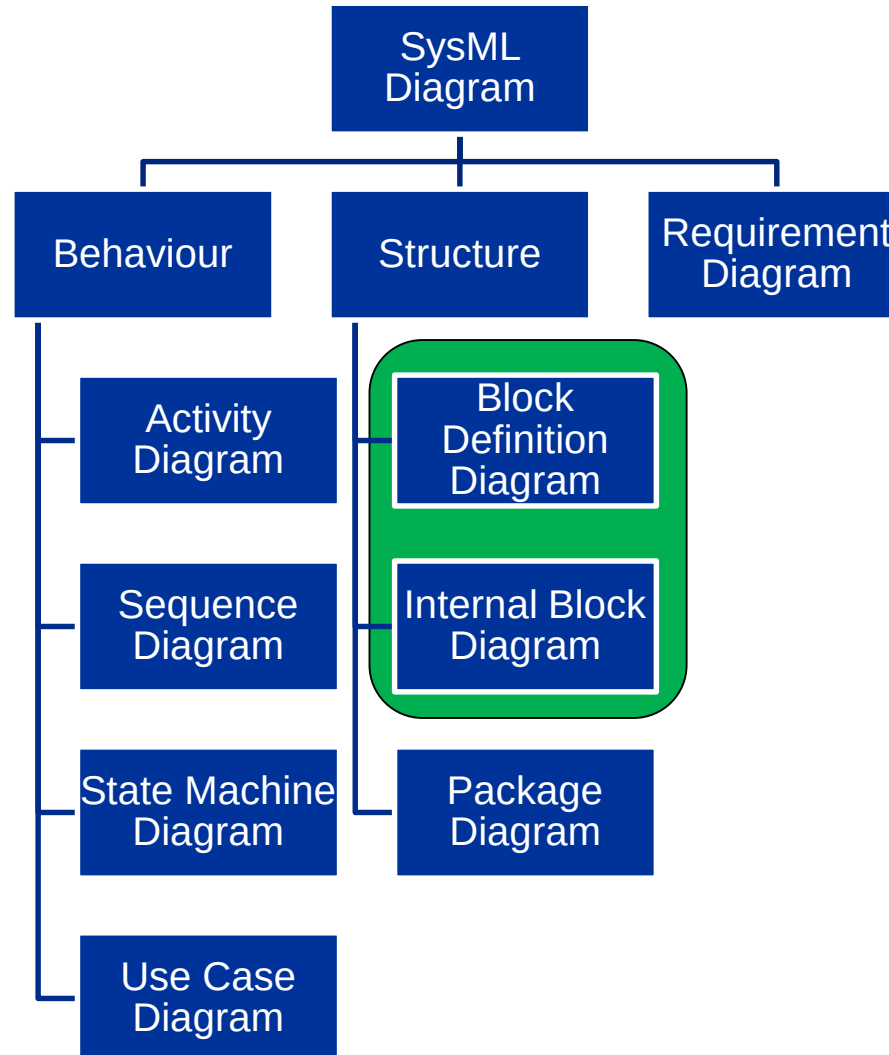


Provide handwritten
conceptual documents (very
likely not so often subject to
change)

Modelling History

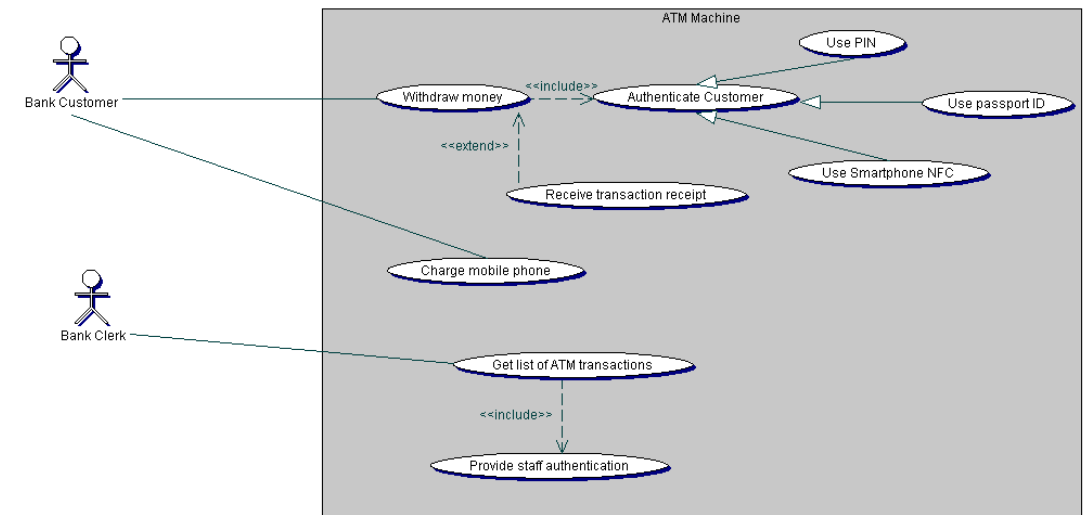


SysML and UML – List of Diagrams



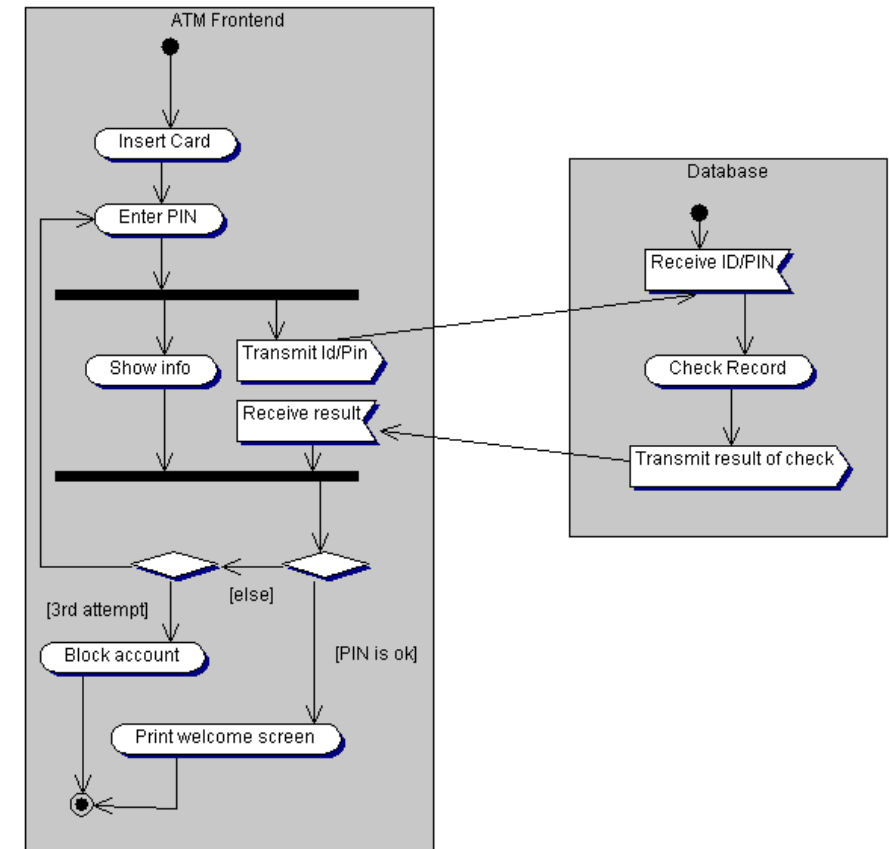
UML Use Case Diagram

- An Use Case Diagram describes the behaviour from an external stakeholder (e.g customer) perspective.
- The question: “What does the stakeholder / actor want to do with the system”
- The answer comes as a Use Case
 - Title as mini sentence
 - Additional description
 - Pre- and postconditions
 - Normal and exceptional behaviour
- Extend (MAY), Include (MUST) and Generalisation relations may be used.



Activity Diagram

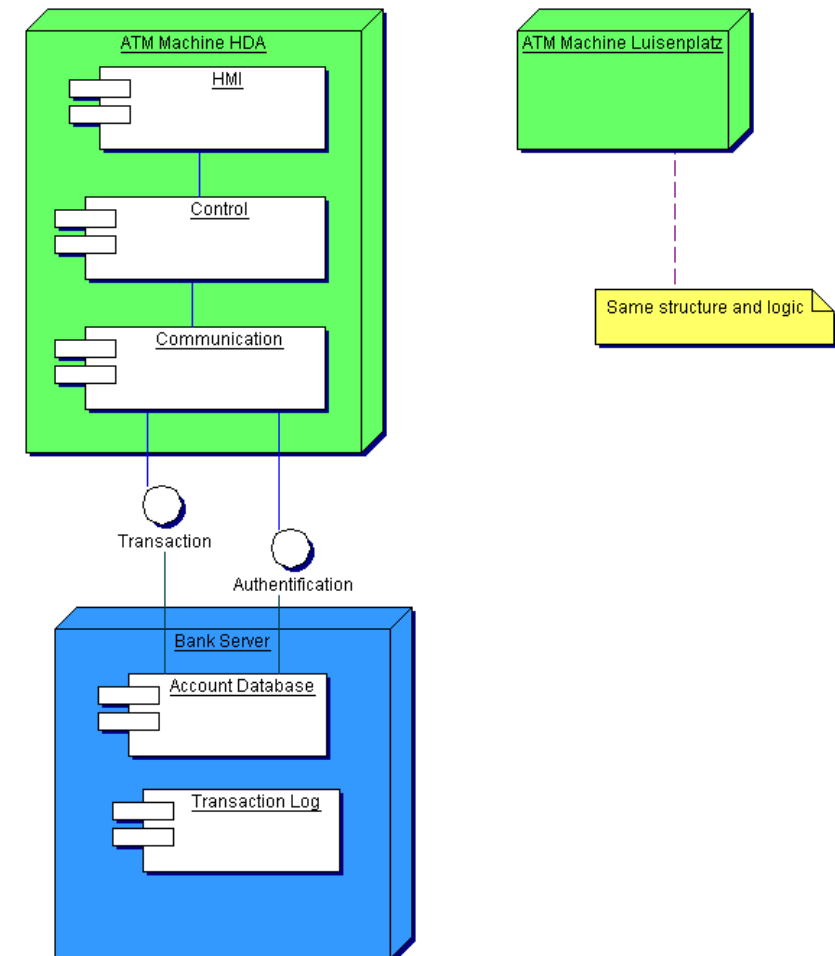
- Shows a sequence of operations.
- Can be used on various levels, overview down to detail.
- Default elements:
 - Start and Endpoint
 - Activities
 - Decision Points and Guards
- Extended features
 - Swim lanes for system boundaries (e.g. technical system and user or different components or processes)
 - Send / Receive blocks for inter process communication
 - Join/Fork bars for parallel processes



KISS: Avoid too many details!

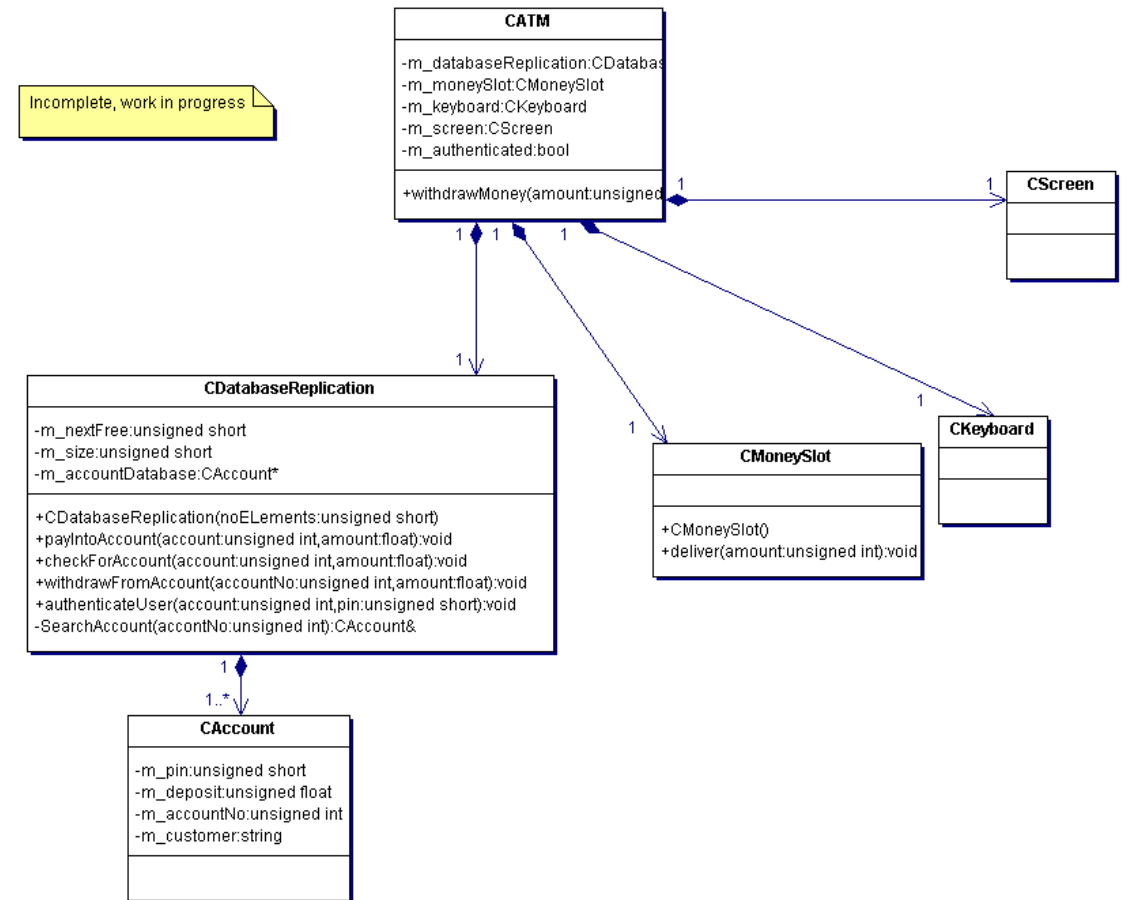
UML Component / Deployment Diagram

- High Level Structural Diagram
- Nodes illustrate hardware components
- Components illustrate software packages
- Interfaces are used to illustrate public services
- Part/Whole relations as well as associations may be used to show internal structures.
- Interfaces are implemented by classes.



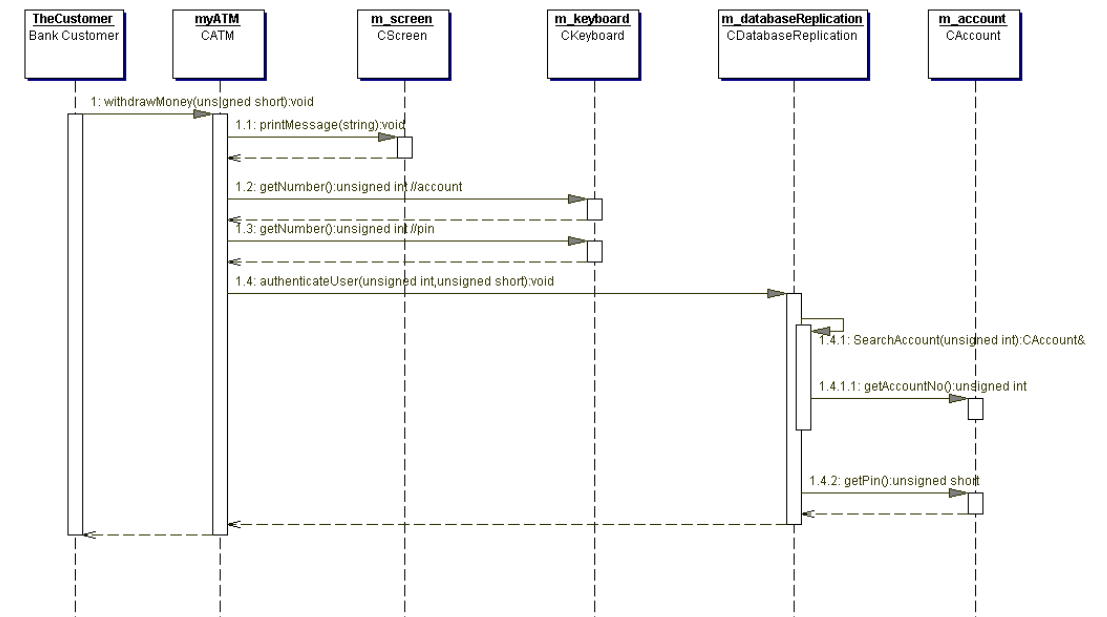
UML Class Diagram

- Probably the most important diagram for OO-architectures.
- Describes class relations and class capabilities.
- Level of detail can be varied.
 - Only classes
 - ...with public methods
 - ...with all methods and attributes
 - ...with method parameters



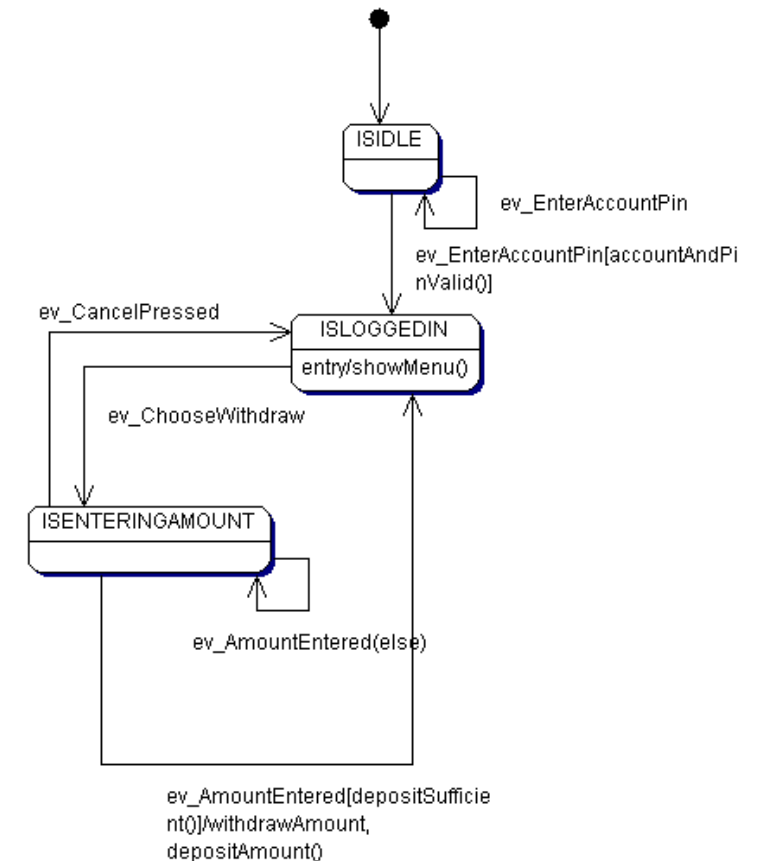
Sequence Diagram

- Shows the timeline of method invocations
- Technical illustration of a single use case at a time.
- Exceptional behaviour requires extra diagram.
- Great for a first validation of the class design (to check if we forgot some methods).



State Diagram

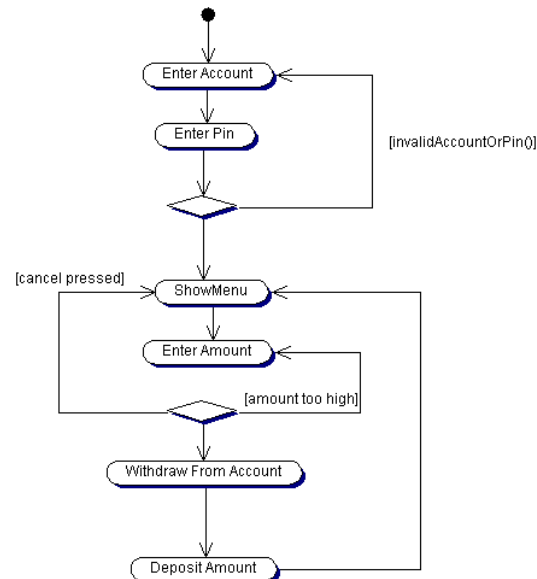
- Describes the system reaction on events.
- Transition: depending on the states, different events may be considered and different activities may be triggered.
- Transition format: event[guard]/action
 - Guard and action are optional
 - Event is mandatory. If you cannot find an event, you have one state too much.



Comparing Activity and State Diagram

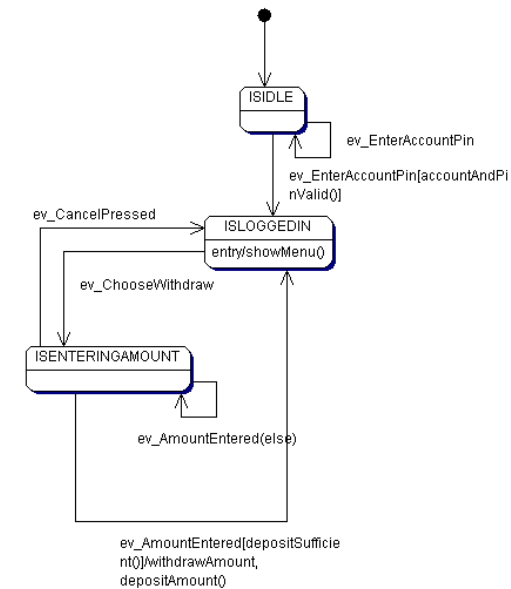
Activity Diagram

- Good for sequential behaviour
- Easier too understand
- Implementation details hard to show (e.g. cancel press)



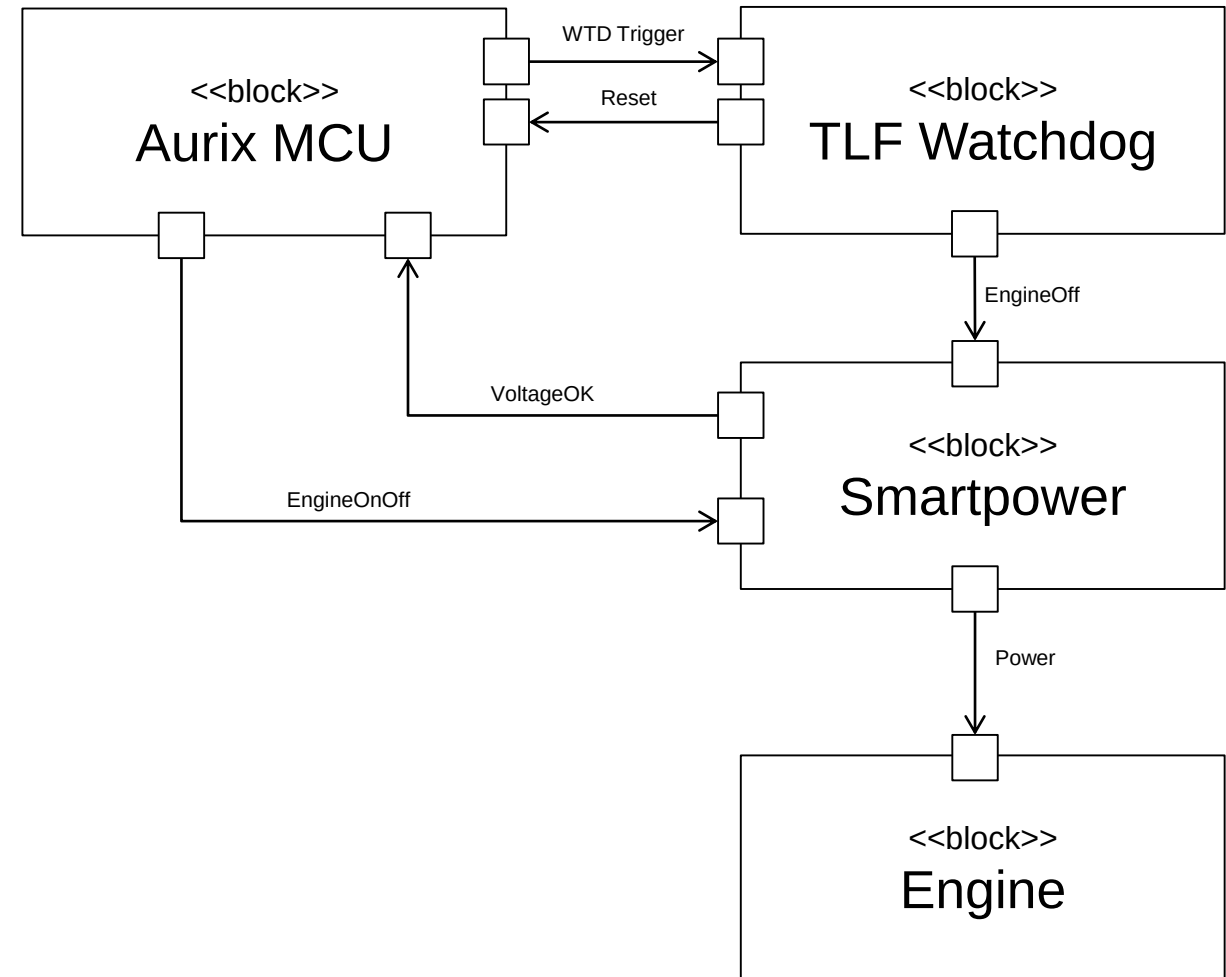
State Diagram

- Good for reactive systems
- Strong technical semantics
- Transitions and States must be found



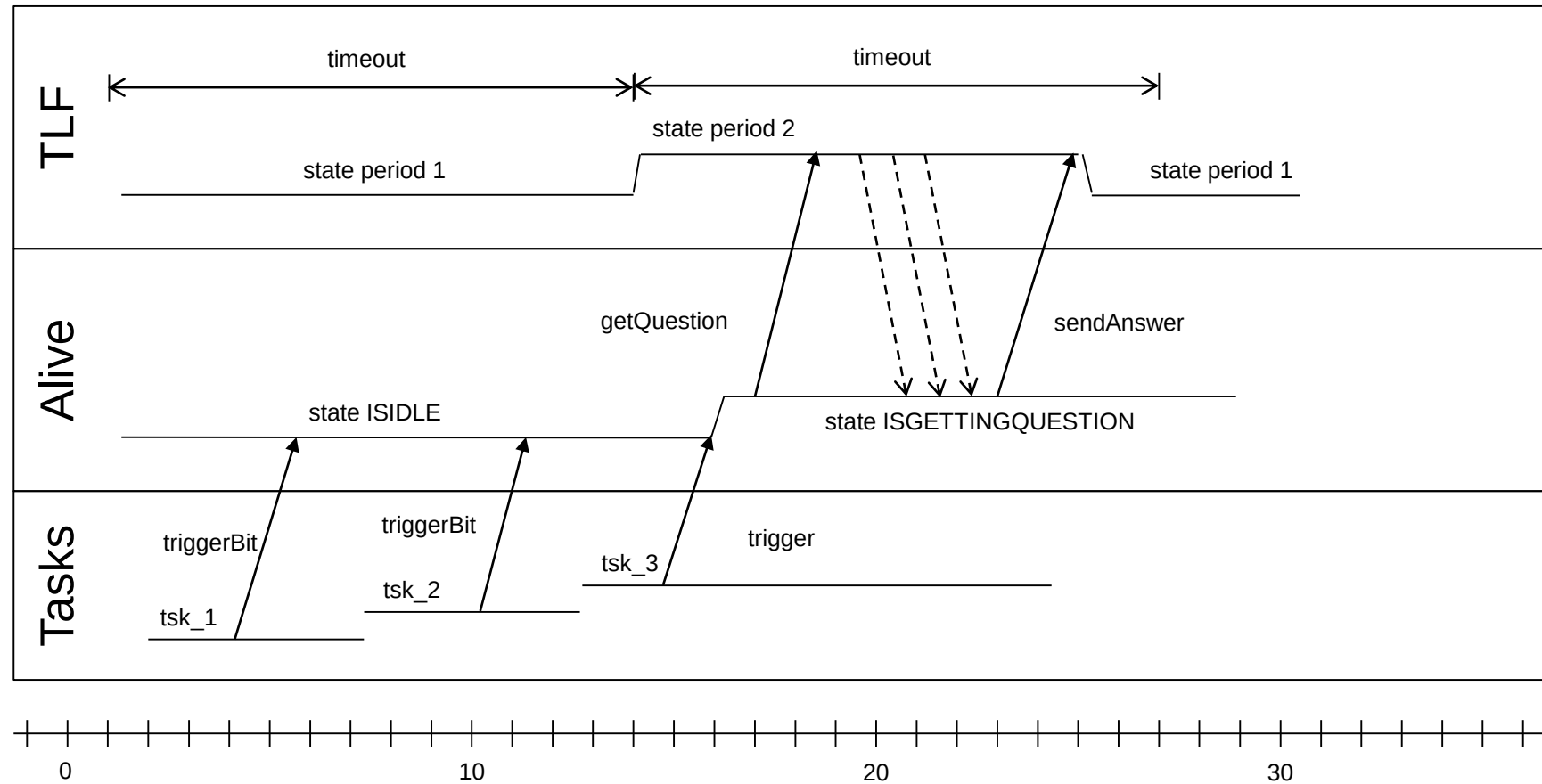
SysML Block Diagram

- Very flexible Diagram focussing on the structure and data exchange.
- Blocks can be be anything, hardware, software, abstract functions.
- Often serves as a starting point for showing and discussing the technical structure of the product.

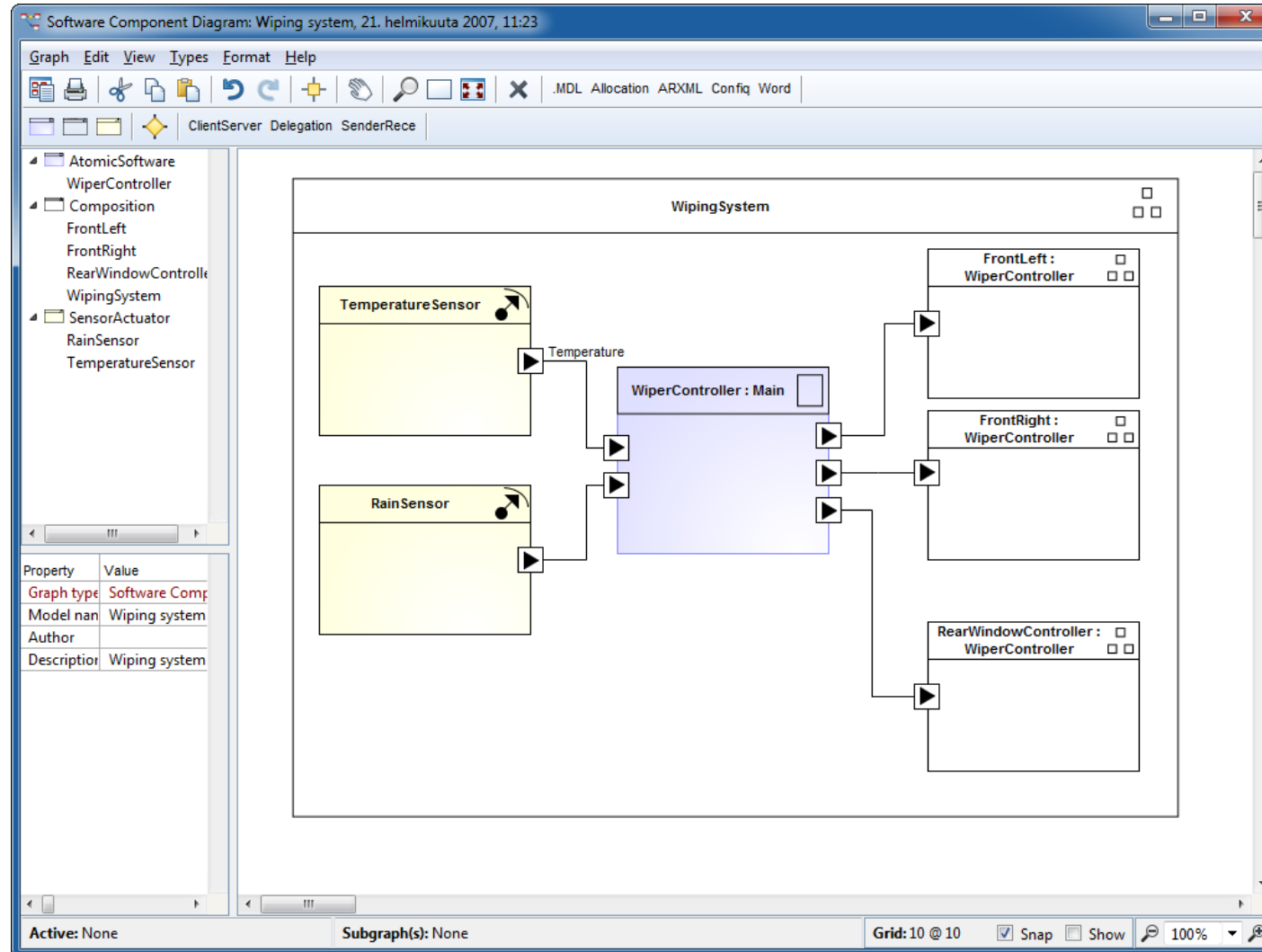


UML Timing Diagram

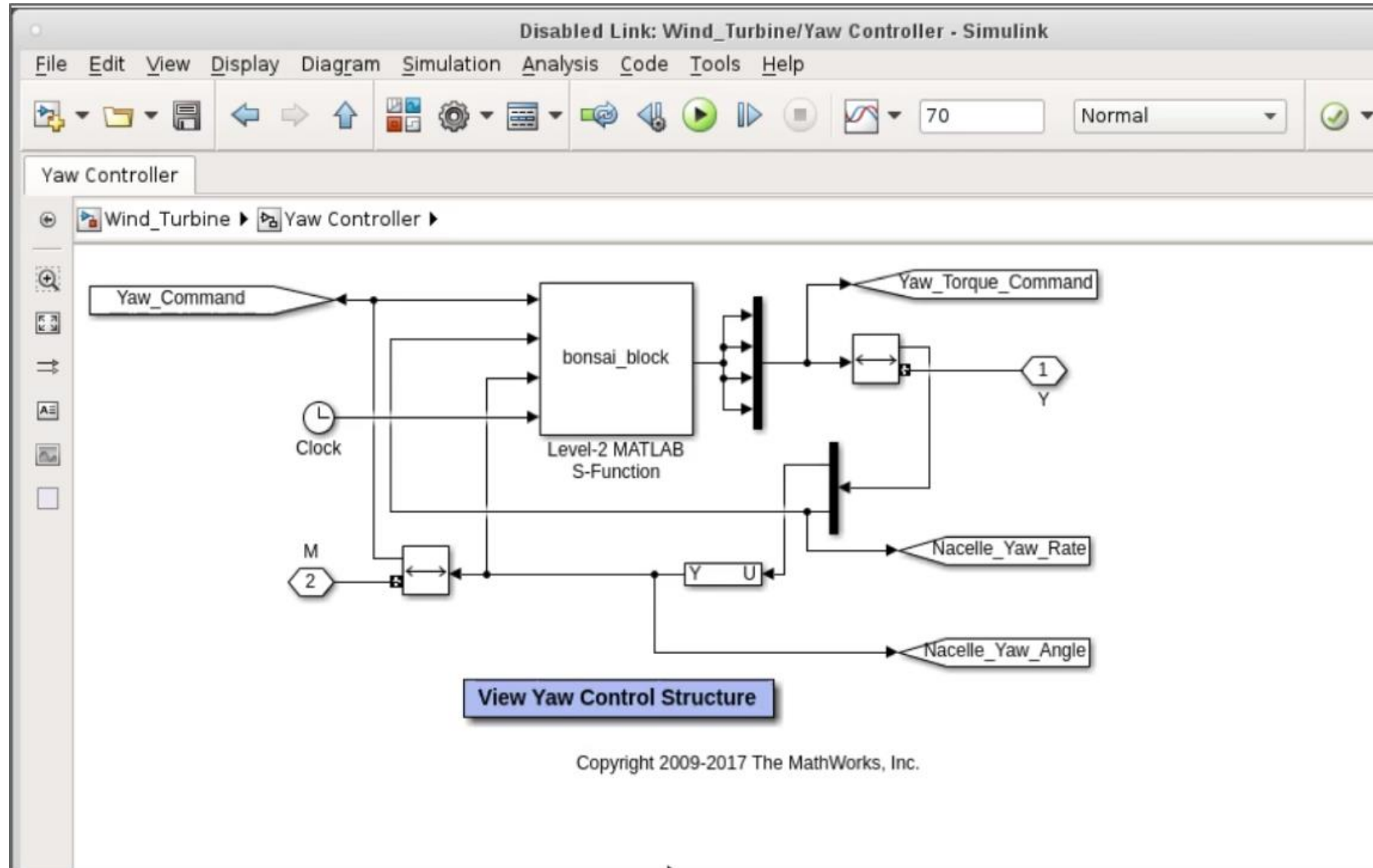
- Explains technical constraints



Autosar



Matlab/Simulink



Rules for Models

- Find the right level of abstraction
 - Too high → no information provided
 - Too many details → user gets lost in space
- Models are an important part of the specification
 - But not the only → prose, conceptual drawings, wireframes, prototypes,...
- Excuse me, do you talk UML?
 - SysML and UML are de-facto standards
 - But there are more domain specific notations, e.g. Autosar, Matlab,...
- Model and code must be in sync!

Diagrams and Abstraction Level

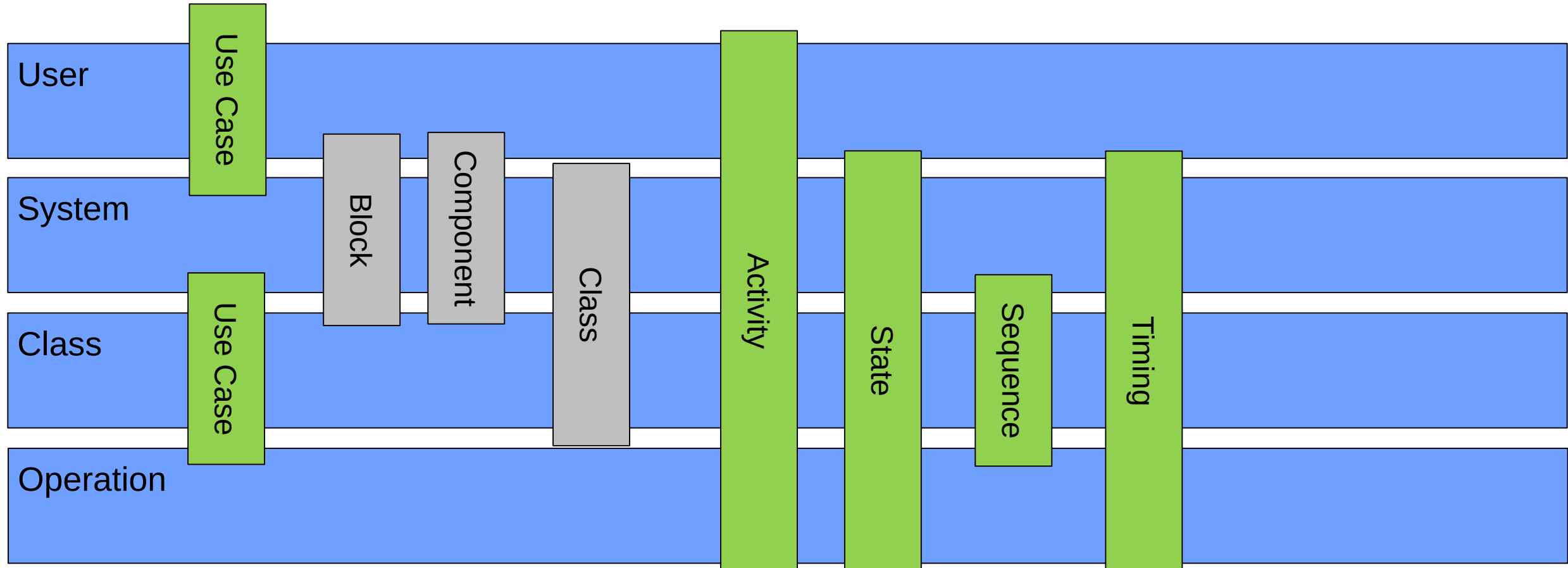
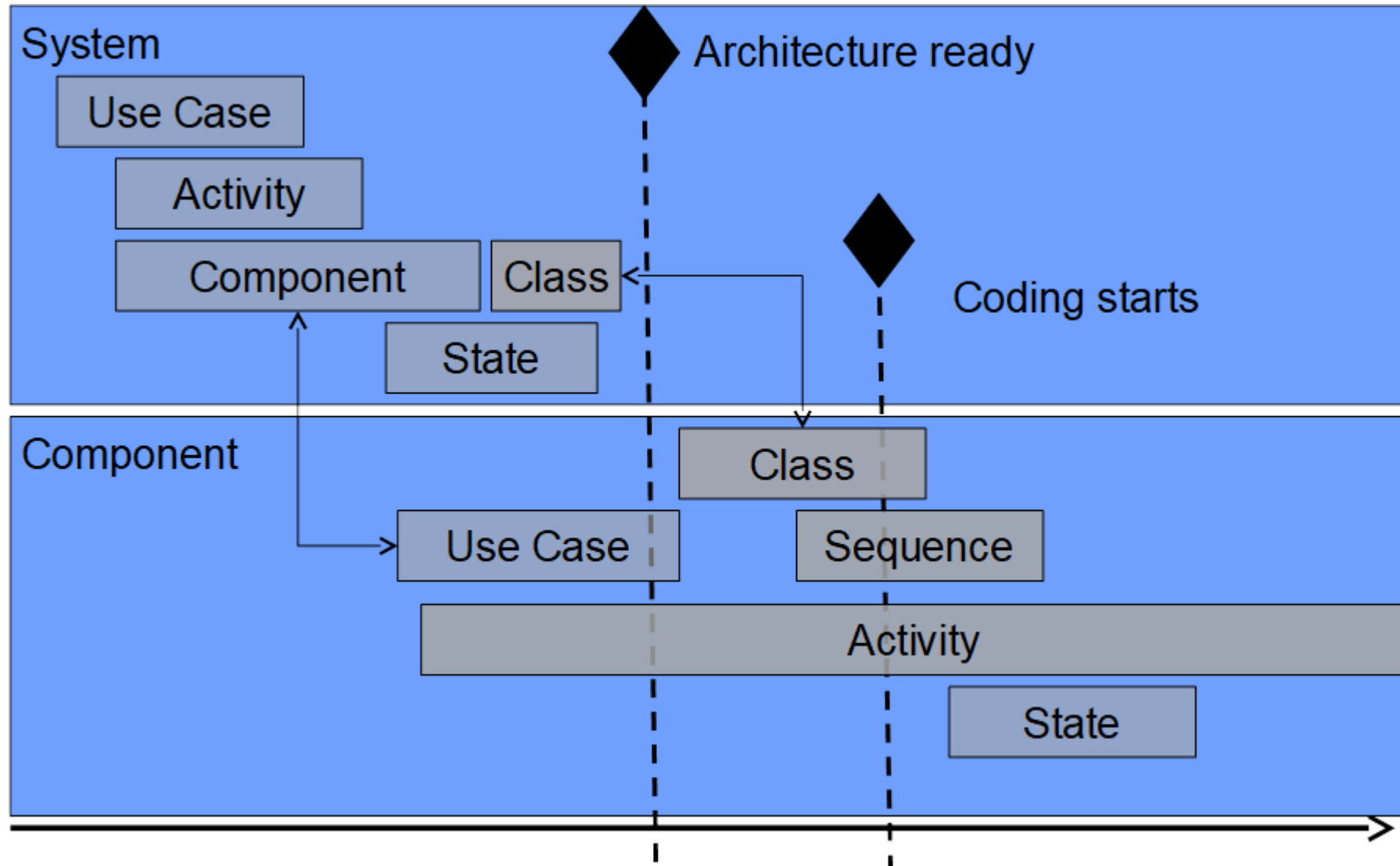


Diagram Workflow



Together and Eclipse

- Together follows the life code concept, i.e. class diagrams, sequence diagrams are based on the real code.
- Create a new Eclipse project
- Copy any existing .tpr file into the mycode folder
- Doubleclick on the .tpr file.
- This will make everything in this folder (and all subfolders) visible for Together
- Use the top level reverse engineering diagram (4 dot diagram) for creating the classes. This will make sure that the classes are created at the right spot
- Create a model package (= folder) for storing the models
- Use Shortcut references to drag the classes on the class diagram.

Example



Code and documentation

Comments do not only help you to understand code, it also help you to

- Understand the requirements
- Structure the code
- Facilitate debugging

As documentation tends to be outdated in no time, we will generate the documentation out of the code using doxygen.

Check <http://www.doxygen.nl/index.html> for details on the syntax.

Check <https://anb0s.github.io/eclox/> for details on the Eclipse integration (section usage)

Install <http://www.graphviz.org/> for graph generation in Doxygen



How to use Doxygen

- Doxygen is already installed in the provided Eclipse package
- You can create new Doxygen projects (also called doxyfiles) using the creation wizard. Go to **“File -> New -> Other -> Other -> Doxyfile”**. Press next and set both file location and name. Then an empty doxyfile will be created at the specified location. The wizard automatically adds the “.Doxyfile” extension.
- Doubleclick on the file to edit the properties
- Rightclick on the project and choose @ Build Documentation
- Open the index.html file created in the html subfolder (press F5 if it is not visible)

Commenting Conventions

Class

- The header file contains the visible API for the user.
- Add comments for the class, describing the class behaviour.
- Additional tags
 - `\author` for your name and contact
 - `\todo` for open issues
 - `\warning` for critical constraints
 - `\note` for special usage hints
 - `\version` for a version history

Class Documentation

```
/**
 * This container class stores a full 360deg scan.
 * It is implemented as a non-overwriting ringbuffer (i.e.
 *   FIFO).
 *
 * \author peter.fromm@h-da.de
 *
 * \note
 * - none
 *
 * \warning No error handling implemented yet
 *
 * \todo
 * - Add error handling at various places
 * - Implement Copy Constructor and Assignment Operator
 *
 * \version
 * - 25.10.19 - Initial version
 * - 26.10.19 - Added comments
 */
class CScan
{
}
```

Detailed Description

This container class stores a full 360deg scan. It is implemented as a non-overwriting ringbuffer (i.e. FIFO).

Author

peter.fromm@h-da.de

Note

- none

Warning

No error handling implemented yet

Todo:

- Add error handling at various places
- Implement Copy Constructor and Assignment Operator

Version

- 25.10.19 - Initial version
- 26.10.19 - Added comments

Attribute and Methods Documentation

- The documentation in the header file describes the responsibility of the methods, attributes as well as the details of the API, like parameters and return values.
- For attributes
 - Describe the attribute above or right of the attribute
 - Often, a `\brief` description will be sufficient
 - Describe the unit and the value range
 - Describe specific error values if needed
- For methods
 - Describing the method's responsibility
 - Provide an additional `\brief` description
 - Describe normal and exceptional behaviour
 - Describe parameters and return values using `\param` and `\return`

Attribute and Methods Documentation

```
class CScan
{
private:

    unsigned short m_readIdx; /**< \brief Current readindex for the ringbuffer. */

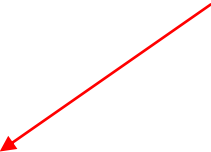
    /**
     * \brief Number of elements currently stored
     *
     * As long as m_fillLevel < m_suze, the buffer is not full and data can be written.
     * If m_fillLevel == 0, the buffer is empty and no data can be read.
     */

public:

    /**
     * \brief Constructor
     *
     * The constructor allocates memory for the dynamic array.
     * Currently, no error handling is implemented, we assume that we have enough memory.
     * \param unsigned short size = 360 : IN - Size of the array to be allocated
     */

    CScan(unsigned short size = 360);

};
```



Attribute and Methods Documentation

Public Member Functions

	CScan (unsigned short size=360) Constructor. More...
virtual	~CScan ()
	CScan (const CScan &other)
void	writeCoordinate (CPolarCoordinate data)
CPolarCoordinate	readCoordinate ()
void	clear ()

Private Attributes

unsigned short	m_size
unsigned short	m_readIdx Current readindex for the ringbuffer. More...
unsigned short	m_writedx
unsigned	m_fillLevel Number of elements currently stored. More...
CPolarCoordinate *	m_pPolarCoordinates

◆ CScan() [1/2]

CScan::CScan (unsigned short size = 360)

Constructor.

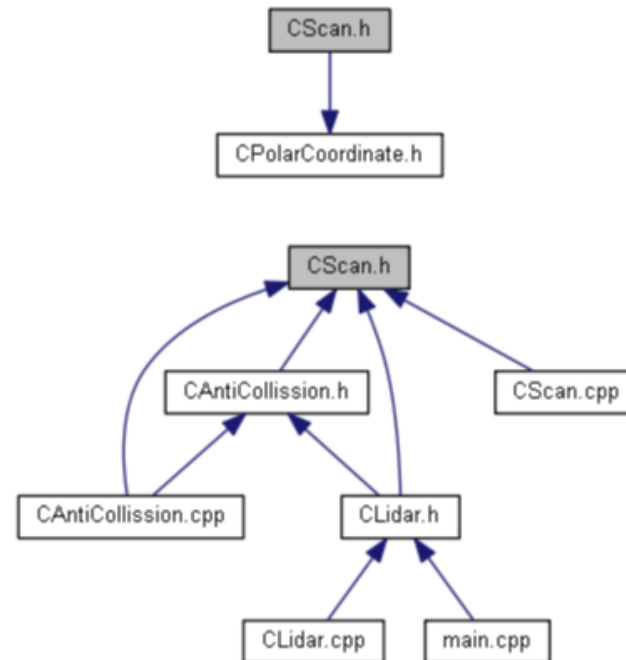
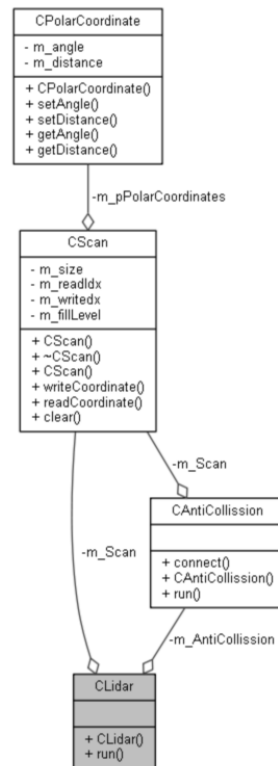
The constructor allocates memory for the dynamic array. Currently, no error handling is implemented, we assume that we have enough memory.

Parameters

unsigned short size = 360 : IN - Size of the array to be allocated

GraphViz

- Install the GraphViz package if you want to create graphical representations of your file.
- Activate the package in the options and set the advanced options as required



Comments inside the functions

- In addition to the description of the interface, we have to document the code structure.
- This should be done BEFORE writing code!
 - It supports you to structure the code
 - It will ensure that you write comments (as they will never be written afterwards)

```
void CScan::writeCoordinate(CPolarCoordinate data)
{
    //Check if array has space

    //Space available

    //Write data to next location

    //Increment writeindex and perform overrun

    //Increment filling level of the array

    //Error handling
}
```

```
void CScan::writeCoordinate(CPolarCoordinate data)
{
    //Check if array has space
    if (m_fillLevel < m_size)
    {
        //Space available

        //Write data to next location
        m_pPolarCoordinates[m_writedx] = data;

        //Increment writeindex and perform overrun
        m_writedx++;
        m_writedx %= m_size;

        //Increment filling level of the array
        m_fillLevel++;
    }
    else
    {
        //todo: add some error handling
    }
}
```

Wrap-Up

- Complete overview of UML 1.4 plus a peek into SysML and UML 2.x
- Modelling workflow
- Doxygen